


**Budapest University of Technology and Economics**
Faculty of Electrical Engineering and Informatics
Department of Networked Systems and Services

# Simulation Framework for Comparing Surface Codes

Bachelor's Thesis

*Author*
Akito Yamaguchi

*Advisor*
dr. Solymos Balázs
Botond Márton

October 29, 2026

# Contents

**Abstract** **i**

**1 Introduction** **1**

1.1 Motivation . . . 1

1.2 Problem Statement and Research Questions . . . 2

1.3 Approach and Contributions . . . 2

1.4 Scope . . . 3

1.5 Thesis Outline . . . 3

**2 Background** **4**

2.1 Qubits and Quantum Circuits . . . 4

2.1.1 Qubits . . . 4

2.1.2 Quantum Circuits . . . 5

2.2 The Paulis and Clifford Groups . . . 5

2.2.1 Pauli Groups . . . 5

2.2.2 Clifford Groups . . . 6

2.3 Noise in Quantum Systems . . . 7

2.3.1 Error Sources and Their Discretization . . . 7

2.3.2 Pauli Channels . . . 8

2.4 Stabilizer Codes . . . 8

2.4.1 Fundamental Idea . . . 8

2.4.2 Stabilizer Formalism . . . 9

2.4.3 The Three-Qubit Repetition Code . . . 9

2.5 The Surface Code . . . 10

2.5.1 The Rotated Surface Code . . . 10

2.5.2 Syndrome Extraction . . . 11

2.6 Decoding . . . 13

2.6.1 The Decoding Problem . . . 13

2.6.2 Error Chains . . . 13

2.6.3 Decoders . . . 14
2.7 Clifford Deformed Surface Codes . . . 14
2.7.1 Definition . . . 14
2.7.2 Biased Noise and Necessity of Clifford Deformation . . . 15
2.8 Memory Experiment and Quantifying Results . . . 16
2.8.1 Memory Experiment . . . 16
2.8.2 Logical Error Rate . . . 17
2.8.3 Threshold . . . 17
2.8.4 Suppression and Teraquop Footprint . . . 18

**3 Design and Implementation** **19**

**4 Evaluation** **20**
4.1 Depolarizing Code-Capacity Threshold . . . 20
4.2 Phenomenological Threshold against Stim . . . 20
4.3 Phenomenological Threshold under Biased Noise . . . 20
4.4 Depolarizing Circuit-Level Threshold . . . 20
4.5 Below-Threshold Suppression and Teraquop Footprint . . . 20
4.6 Decoder Comparison at Code Capacity . . . 20

**5 Discussion** **21**

**6 Conclusion and Future Work** **22**

**Acknowledgements** **23**

**Bibliography** **24**

## STUDENT DECLARATION

I, *Akito Yamaguchi*, the undersigned, hereby declare that the present Bachelor's Thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized. In the Appendix, I have clearly indicated whether I used artificial intelligence tools in preparing the thesis; if so, I have specified the manner and extent of their use in the table. I acknowledge that I bear full responsibility for any content generated by artificial intelligence – regardless of the extent of its use.

I agree that the basic information of my present work (author(s), title, abstracts in English and in a second language, year of preparation, name of advisor(s)) will be made publicly available on the internet by BME VIK, and the full text of the work will be available to persons registered in the BME Directory. I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, October 29, 2026

*Akito Yamaguchi*
student

# Abstract

# Chapter 1

# Introduction

## 1.1 Motivation

Quantum computers process information in a fundamentally different way from classical computers. Their qubits can be in superpositions of 0 and 1 and can be entangled with each other (Section 2.1), and quantum algorithms exploit interference between computational paths to solve certain problems much faster than any known classical algorithm. Shor's algorithm factors integers in polynomial time [49], Grover's algorithm searches an unstructured list with quadratically fewer queries [30], and, as Feynman suggested, quantum computers can simulate quantum systems in chemistry and materials science that are intractable for classical computers [20, 42]. These advantages are specific to particular problems, however; a quantum computer is not a faster general-purpose computer. Its main disadvantage is fragility. Classical hardware operates practically without errors, whereas the best physical qubits today fail with a probability of about $10^{-3}$ per operation [26], which limits present-day devices to short, noisy computations [46]. Useful algorithms, such as factoring a 2048-bit RSA modulus, require billions of logical operations [23], so the error rate per operation has to drop from about $10^{-3}$ to $10^{-12}$ or below [24].

Quantum error correction (QEC) closes this gap by encoding one logical qubit in many physical qubits and correcting errors faster than they accumulate [48, 53]. Its leading candidate is the surface code [39, 15, 21]: it requires only nearest-neighbour interactions on a two-dimensional grid and tolerates comparatively high error rates, and operation below its threshold has recently been demonstrated on superconducting hardware [26]. Its price is overhead: reaching the error rates of useful algorithms takes on the order of a thousand physical qubits per logical qubit [21, 24].

This overhead is usually estimated for noise that treats all types of errors alike, but in many hardware platforms errors are not symmetric. Dephasing, which causes $Z$ errors, often dominates [2, 55], and in cat qubits the bias is engineered on purpose, so that bit flips are suppressed exponentially [47]. A code tailored to such biased noise can reach the same protection with fewer qubits [4, 12]. Clifford deformations offer an inexpensive way to tailor the surface code [16]: applying single-qubit Clifford operations to the data qubits changes which Pauli operator each check measures on each qubit, while the lattice, the qubit count and the measurement schedule stay the same (Section 2.7). The XZZX code [4] and the XY code [55] are the best-known examples.

## 1.2 Problem Statement and Research Questions

The family of Clifford-deformed surface codes is large: each of the $n$ data qubits can be deformed in six ways, giving $6^n$ codes, of which $3^n$ remain inequivalent under $Z$-biased noise with $p_X = p_Y$ (Section 2.7.2). Dua et al. studied small patches and random ensembles of this family [16]; beyond that, only a few members, notably the XZZX and XY codes, have been studied in depth [56, 57, 4, 12, 35]. Moreover, the published results for these codes were obtained with different boundary conditions, noise models, noise conventions and decoders, so they cannot be compared directly.

Studying a new deformation at the circuit level is laborious. One has to build its syndrome-extraction circuit, decide which detectors are valid at the time boundaries, check that the gate schedule preserves the distance, and run a complete sampling and analysis pipeline. Fast stabilizer simulators such as Stim [22] make the sampling cheap, but they generate circuits only for a few standard codes and leave the construction and verification of all other circuits to the user. Done by hand for each code, every step is a source of silent errors: an omitted detector or a poorly chosen gate order does not crash a simulation, but quietly degrades its results. What is missing is a way to define an arbitrary deformation once, verify that its circuits are correct, and compare it with other codes under identical conditions. This thesis therefore addresses three research questions:

**RQ1 (Design).** Can the circuits of any Clifford deformation of the surface code, and more generally of a static stabilizer code, at the code-capacity, phenomenological and circuit levels, be generated automatically from a definition in the stabilizer formalism, and how can a code and its generated circuits be verified as correct before sampling?

**RQ2 (Reproduction).** Do simulations with the framework reproduce published results for known codes at each of the three noise levels?

**RQ3 (Usefulness).** Does the framework make it practical to add a new deformation and to compare it with existing codes under identical conditions?

## 1.3 Approach and Contributions

To answer these questions, this thesis develops a simulation framework for Clifford-deformed codes, whose design is presented in Chapter 3. A code is declared by its stabilizer generators, its logical operators, its syndrome-extraction schedule and the deformation of each data qubit. The framework validates the declaration, builds the memory-experiment circuits, samples and decodes them, and estimates the logical error rate, the threshold, the error suppression factor and the teraquop footprint (Section 2.8). A single configuration describes an entire study and suffices to reproduce it.

The contributions of this thesis are:

- a framework that builds memory-experiment circuits at three noise levels for any Clifford deformation of surface codes and, more generally, of static stabilizer codes, from a definition in the stabilizer formalism;
- a validation method that combines algebraic checks on the code with circuit-level checks on its detectors and distance;

- an automated analysis that estimates the threshold, the error suppression factor and the teraquop footprint of every code with the same established methods, so that the results of different codes are directly comparable;
- a case study that compares the CSS code, the XZZX code and partially deformed codes under identical conditions;
- an open-source implementation that records the configuration and the software environment of every run for reproducibility.

## 1.4 Scope

The thesis is limited to static codes, whose checks are the same in every round, since this is what allows the detectors to be generated automatically. Each code encodes a single logical qubit, which keeps comparisons between codes fair, on a two-dimensional lattice, which matches the planar, nearest-neighbour connectivity of current hardware. Codes are deformed only by single-qubit Cliffords on their data qubits, which leave the hardware layout unchanged. The noise is Pauli noise, which stabilizer simulators can sample efficiently (Section 2.2.2); the experiments are memory experiments, the standard benchmark of how well a code stores information; and the decoders are matching-based, since they are fast enough for the millions of shots that a threshold study requires. Out of scope are Floquet and other dynamic codes [31, 24], whose checks change from round to round; logical operations such as lattice surgery [36], which build on a working memory; optimal maximum-likelihood decoders [5], which are too slow for circuit-level noise at scale; and coherent, non-Pauli noise, which stabilizer simulators cannot represent and which randomized compiling converts into Pauli noise [58].

## 1.5 Thesis Outline

Chapter 2 introduces the background: qubits and noise models, stabilizer codes, the surface code and its decoding, Clifford deformations and the measures of code performance. Chapter 3 presents the design of the framework, including its validation method, together with its implementation. Chapter 4 evaluates the generation and validation of the circuits, the reproduction of published results and, through a case study, the extensibility of the framework, answering RQ1–RQ3. Chapter 5 discusses the results and their limitations, and Chapter 6 concludes and outlines future work.

# Chapter 2

# Background

This chapter summarizes the theory on which the rest of the thesis builds. Sections 2.1–2.3 introduce qubits and quantum circuits, the Pauli and Clifford groups, and the noise models used throughout. Sections 2.4–2.6 develop the stabilizer formalism, the rotated surface code with its syndrome-extraction circuit, and the decoding problem. Section 2.7 defines the Clifford-deformed surface codes compared in this work, including the XZZX code, and Section 2.8 describes the memory experiment together with the figures of merit used to evaluate the codes: the logical error rate, the threshold, the error suppression factor and the teraquop footprint. The presentation favours intuition over proofs; derivations can be found in the cited references.

## 2.1 Qubits and Quantum Circuits

### 2.1.1 Qubits

The basic unit of quantum information is the *qubit*. Its state is a unit vector in the two-dimensional complex Hilbert space $\mathbb{C}^2$ [44, Sec. 1.2, 2.2]. In the *computational basis* $\{|0\rangle, |1\rangle\}$, a general state reads

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \tag{2.1}$$

Whereas a classical bit is either 0 or 1, a qubit can be in a *superposition* of both basis states; the information is carried by the complex amplitudes $\alpha$ and $\beta$, including their relative phase. A second basis used frequently in this thesis is the $X$ *basis* $\{|+\rangle, |-\rangle\}$ with $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$.

A register of $n$ qubits lives in the tensor product $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$, a space of dimension $2^n$. Its basis consists of the $2^n$ computational-basis states $|x_1 x_2 \cdots x_n\rangle$ with $x_i \in \{0, 1\}$, so a general $n$-qubit state is specified by $2^n$ complex amplitudes. This exponential growth is the source both of the potential power of quantum computers and of the difficulty of simulating them classically. A state of a composite system that can be written as a product, such as $|\psi_1\rangle \otimes |\psi_2\rangle$, is a *product state*; states that do not factor in this way are *entangled*. The standard example is the Bell state

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}, \tag{2.2}$$

for which measuring either qubit yields a uniformly random bit, yet the two outcomes always agree. Quantum error-correcting codes use entanglement to spread the information of one logical qubit over many physical qubits.

### 2.1.2 Quantum Circuits

A quantum computation is described by a *quantum circuit*: a sequence of gates acting on a register of qubits, interleaved with or followed by measurements. Quantum gates are unitary operators, $U^\dagger U = I$, so every gate is reversible, and its inverse $U^\dagger$ is again a valid gate [44, Sec. 2.2.2]. Important single-qubit gates, besides the Pauli gates of Section 2.2.1, are the Hadamard gate $H$ and the phase gate $S$,

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \tag{2.3}$$

where $H$ maps the computational basis onto the $X$ basis, $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$. A *controlled-$U$* gate applies $U$ to a target qubit if and only if its control qubit is in state $|1\rangle$, i.e., it acts as $|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$. The most common two-qubit gate is the controlled-NOT gate (CNOT or CX), which flips the target conditioned on the control; the controlled-$Y$ and controlled-$Z$ gates (CY and CZ) are defined analogously, and all three appear in the syndrome-extraction circuits of Section 2.5.2.

Measurement, together with qubit reset, which can be viewed as a measurement followed by a conditional bit flip, is the only irreversible element of a circuit [44, Sec. 2.2.3, 4.4]. Measuring the state (2.1) in the computational basis returns outcome 0 with probability $|\alpha|^2$ and outcome 1 with probability $|\beta|^2$ (the Born rule), and the state *collapses* onto the basis state that corresponds to the outcome; the amplitudes $\alpha$ and $\beta$ are lost. A measurement in another basis rotates that basis onto the computational one first; an $X$-basis measurement, for instance, is a Hadamard gate followed by a computational-basis measurement. More generally, measuring a Pauli operator (Section 2.2.1) returns one of its eigenvalues $\pm 1$ and projects the state onto the corresponding eigenspace.

Gates acting on disjoint qubits can be executed in parallel, so a circuit is organized into *time steps* (layers), and within one time step each qubit takes part in at most one gate, measurement or reset. This seemingly technical constraint turns the order of the gates in syndrome-extraction circuits into a scheduling problem (Section 2.5.2).

## 2.2 The Paulis and Clifford Groups

### 2.2.1 Pauli Groups

The single-qubit Pauli operators are

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2.4}$$

$X$ exchanges $|0\rangle$ and $|1\rangle$ and is called a *bit flip*, $Z$ multiplies $|1\rangle$ by $-1$ and is called a *phase flip*, and $Y = iXZ$ does both [44, Sec. 2.1.3]. The Pauli matrices are Hermitian and unitary, square to the identity, and multiply as $XY = iZ$, $YZ = iX$ and $ZX = iY$. Together with the phases $\pm 1$ and $\pm i$, which are needed for closure under multiplication, they form the single-qubit *Pauli group* $\mathcal{P}_1 = \{\pm I, \pm iI, \pm X, \pm iX, \pm Y, \pm iY, \pm Z, \pm iZ\}$ [44,

**Table 2.1:** The six single-qubit Clifford operations modulo Pauli operators and phases, written as permutations of $X$, $Y$ and $Z$ under the conjugation $P \mapsto CPC^\dagger$ (signs omitted). Here $\sqrt{X} = HSH$.

| $C$ | $X \mapsto$ | $Y \mapsto$ | $Z \mapsto$ | Action |
|---|---|---|---|---|
| $I$ | $X$ | $Y$ | $Z$ | identity |
| $H$ | $Z$ | $Y$ | $X$ | $X \leftrightarrow Z$ |
| $S$ | $Y$ | $X$ | $Z$ | $X \leftrightarrow Y$ |
| $\sqrt{X}$ | $X$ | $Z$ | $Y$ | $Y \leftrightarrow Z$ |
| $HS$ | $Y$ | $Z$ | $X$ | cycle $X \to Y \to Z \to X$ |
| $SH$ | $Z$ | $X$ | $Y$ | cycle $X \to Z \to Y \to X$ |

Sec. 10.5.1]. Any two Pauli operators either commute or anticommute: any two of $X$, $Y$ and $Z$ anticommute, e.g., $XZ = -ZX$, while every Pauli commutes with itself and with $I$.

The $n$-qubit Pauli group consists of all $n$-fold tensor products of single-qubit Paulis together with an overall phase:

$$\mathcal{P}_n = \left\{ i^k \, P_1 \otimes P_2 \otimes \cdots \otimes P_n \ : \ k \in \{0,1,2,3\}, \ P_j \in \{I,X,Y,Z\} \right\}. \tag{2.5}$$

The tensor product symbol is usually omitted, as in $XZZX = X \otimes Z \otimes Z \otimes X$, and $X_j$ denotes $X$ acting on qubit $j$. Because the single-qubit factors commute or anticommute independently of each other, two $n$-qubit Pauli operators anticommute if and only if they anticommute on an odd number of qubits, and commute otherwise [27]. For example, $XX$ and $ZZ$ commute because they anticommute on two qubits—the property that makes the checks of the surface code compatible (Section 2.5.1). Up to phases, an $n$-qubit Pauli $P \propto X^{\mathbf{x}} Z^{\mathbf{z}}$ can be stored as a binary vector $(\mathbf{x}|\mathbf{z}) \in \{0,1\}^{2n}$, and the commutation rule becomes the symplectic inner product $\mathbf{x} \cdot \mathbf{z}' + \mathbf{z} \cdot \mathbf{x}' \bmod 2$ [9]; this binary representation is what stabilizer simulators manipulate internally.

The *weight* of a Pauli operator is the number of qubits on which it acts non-trivially, i.e., with a factor other than $I$; for instance, $XIZY$ has weight three. Weights measure both the size of an error and the size of a check of a code.

### 2.2.2 Clifford Groups

The *Clifford group* $\mathcal{C}_n$ is the normalizer of the Pauli group: it consists of the unitaries that map Pauli operators to Pauli operators under conjugation [28], [44, Sec. 10.5.2],

$$\mathcal{C}_n = \left\{ U \ : \ UPU^\dagger \in \mathcal{P}_n \text{ for all } P \in \mathcal{P}_n \right\}. \tag{2.6}$$

Since conjugation preserves products and commutation relations, a Clifford unitary is determined, up to a global phase, by the images of the operators $X_j$ and $Z_j$. On a single qubit, a Clifford gate must therefore map the three operators $X$, $Y$ and $Z$ onto each other, up to signs. Modulo Pauli operators and phases, a single-qubit Clifford is thus simply a *permutation of the three Pauli axes*. The $3! = 6$ such permutations are listed in Table 2.1; the Hadamard gate, for example, exchanges $X$ and $Z$, and the phase gate exchanges $X$ and $Y$. These six relabelings are the building blocks of the Clifford-deformed codes in Section 2.7.

Up to global phases, the whole Clifford group $\mathcal{C}_n$ is generated by $H$ and $S$ on every qubit together with CNOT on every pair of qubits [44, Sec. 10.5.2]. With control $c$ and target $t$, CNOT conjugates $X_c \mapsto X_c X_t$ and $Z_t \mapsto Z_c Z_t$, while $Z_c$ and $X_t$ are left unchanged. This is how Pauli errors propagate through two-qubit gates, which will matter for hook errors in Section 2.5.2. Since $\mathrm{CY} = (I \otimes S)\,\mathrm{CX}\,(I \otimes S^\dagger)$ and $\mathrm{CZ} = (I \otimes H)\,\mathrm{CX}\,(I \otimes H)$, all controlled-Pauli gates are Clifford gates as well.

The *Gottesman–Knill theorem* states that circuits consisting only of computational-basis preparations, Clifford gates, Pauli measurements and classical control can be simulated efficiently on a classical computer [28], [44, Sec. 10.5.4]. Instead of the $2^n$ amplitudes of the state, the simulation tracks the $n$ Pauli operators that stabilize it (Section 2.4.2), and every Clifford gate merely maps these Paulis to other Paulis, by rules like the one for CNOT above. The tableau algorithm of Aaronson and Gottesman performs each gate in $O(n)$ and each measurement in $O(n^2)$ time [1], and the stabilizer simulator Stim [22] samples millions of shots of large error-correction circuits by additionally propagating sampled Pauli errors through the circuit as a *Pauli frame*. The circuits studied in this thesis, syndrome-extraction circuits subject to Pauli noise (Section 2.3), are of exactly this type. The Gottesman–Knill theorem is therefore what makes their simulation possible at all: a state-vector simulation of even a distance-11 surface code, which uses 241 qubits, would be far out of reach.

## 2.3 Noise in Quantum Systems

### 2.3.1 Error Sources and Their Discretization

Physical qubits are never perfectly isolated from their environment, nor perfectly controlled. In current hardware, errors arise from several sources [40], [44, Ch. 8]: *decoherence* due to the interaction with the environment, characterized by the energy-relaxation time $T_1$ (decay from $|1\rangle$ to $|0\rangle$) and the dephasing time $T_2$ (loss of the relative phase between $|0\rangle$ and $|1\rangle$); *imperfect gates*, caused by miscalibrated control pulses and unwanted couplings; *faulty readout*, which reports the wrong measurement outcome; and errors accumulated while a qubit *idles*, waiting for operations on other qubits to complete.

All of these processes are continuous; a slightly miscalibrated pulse, for example, applies a small unwanted rotation $e^{-i\theta X/2}$ by some angle $\theta$. A correction procedure, on the other hand, must make a discrete choice of which operation to apply. The two are reconciled by expanding an arbitrary single-qubit error operator $E$ in the Pauli basis,

$$E = e_I\, I + e_X\, X + e_Y\, Y + e_Z\, Z, \tag{2.7}$$

which is possible because the four Pauli matrices form a basis of the complex $2 \times 2$ matrices [44, Sec. 10.3.1]. When the checks of a code are measured (Section 2.4), the corrupted state $E\,|\psi\rangle$ is projected onto one of the terms of (2.7), and the outcomes reveal which one. The continuous error has been *discretized* into a single Pauli error, which is undone by applying the same Pauli operator again. By linearity, a code that corrects a set of Pauli errors corrects every error in their linear span. Moreover, randomized compiling (Pauli twirling) converts general noise into a stochastic Pauli channel while preserving its average fidelity [58]. It therefore suffices to study Pauli noise [29], which is the noise model used throughout this thesis; conveniently, Pauli noise is also exactly the kind of noise that stabilizer simulators can sample efficiently (Section 2.2.2).

### 2.3.2 Pauli Channels

Noise is described by a quantum channel acting on density matrices $\rho$. A single-qubit *Pauli channel* applies $X$, $Y$ or $Z$ with fixed probabilities $p_X$, $p_Y$ and $p_Z$ [44, Sec. 8.3]:

$$\mathcal{N}(\rho) = (1-p)\,\rho + p_X\,X\rho X + p_Y\,Y\rho Y + p_Z\,Z\rho Z, \qquad p = p_X + p_Y + p_Z, \tag{2.8}$$

where $p$ is the total error rate. Two familiar one-parameter special cases are the *bit-flip* channel ($p_X = p$, $p_Y = p_Z = 0$) and the *phase-flip* channel ($p_Z = p$, $p_X = p_Y = 0$). The *depolarizing* channel treats all three errors alike and applies each of $X$, $Y$ and $Z$ with probability $p/3$.

Noise models for error correction differ in where these channels act. In the simplest, *code-capacity* model, noise hits only the data qubits, once, and the checks are measured perfectly. The *phenomenological* model adds noise on the data qubits in every round and flips each measurement outcome with some probability, but still ignores the circuit that performs the measurement [15]. In the most realistic *circuit-level* model, a channel acts after every operation of the circuit: single-qubit channels after single-qubit gates and on idling qubits, flips with some probability on preparations and measurement outcomes, and a two-qubit channel after every two-qubit gate. The latter applies each of the 15 non-identity two-qubit Paulis $P_1 \otimes P_2 \in \{I, X, Y, Z\}^{\otimes 2} \setminus \{II\}$ with its own probability. A two-qubit channel is needed because a faulty gate acts on both of its operands at once and correlates their errors, producing terms such as $XX$ or $ZY$ that independent single-qubit channels cannot describe [60, 21]. The standard choice, two-qubit depolarizing noise, assigns probability $p/15$ to each term.

In many qubit platforms, dephasing processes are much faster than energy relaxation, so $Z$ errors are far more likely than $X$ or $Y$ errors [2]. The strength of such *biased noise* is quantified by the bias [55]

$$\eta = \frac{p_Z}{p_X + p_Y}. \tag{2.9}$$

For $p_X = p_Y$, the individual rates follow from $p$ and $\eta$ as $p_Z = p\,\eta/(1+\eta)$ and $p_X = p_Y = p/(2(1+\eta))$. Depolarizing noise corresponds to $\eta = 1/2$, and the limit $\eta \to \infty$ is pure dephasing, i.e., the phase-flip channel. In a circuit, the bias survives only if the two-qubit gates do not convert $Z$ errors into $X$ or $Y$ errors; such *bias-preserving* gates can be realized, for instance, with stabilized cat qubits [47]. Biased noise is the motivation for the Clifford-deformed codes of Section 2.7.

## 2.4 Stabilizer Codes

### 2.4.1 Fundamental Idea

Classical error correction protects information by redundancy: a bit is copied several times, and errors are corrected by majority vote. Three obstacles seem to rule out the same strategy for quantum information [44, Sec. 10.1]. First, the *no-cloning theorem* forbids copying an unknown quantum state [62]. Second, measuring the qubits to look for errors collapses their state and destroys the superposition that should be protected. Third, errors are continuous, so there are infinitely many of them.

Quantum error correction overcomes all three [48, 50]. Instead of copying the state, it *encodes* the information of a logical qubit into a carefully chosen subspace, the *code space*, of a larger register of physical qubits, spreading it non-locally by entanglement. Instead

of measuring the qubits themselves, it measures *parities*, collective properties of several qubits that reveal whether an error occurred and where, but nothing about the encoded state; the outcomes of these measurements form the *syndrome*. And as discussed in Section 2.3.1, measuring the syndrome projects a continuous error onto a discrete Pauli error. What remains is a classical inference problem, *decoding*: given the syndrome, decide which correction to apply [53]. Optimal decoding is computationally hard in general [38], so practical decoders trade accuracy for speed (Section 2.6).

### 2.4.2 Stabilizer Formalism

The stabilizer formalism describes a large class of quantum codes compactly in terms of Pauli operators [27], [44, Sec. 10.5.1]. A *stabilizer group* $\mathcal{S}$ is an abelian subgroup of $\mathcal{P}_n$ that does not contain $-I$. Rather than listing all of its elements, one specifies a set of *stabilizer generators* $\{S_1, \ldots, S_m\}$, an independent set whose products give the whole group, $\mathcal{S} = \langle S_1, \ldots, S_m \rangle$. The code space is the simultaneous $+1$ eigenspace of all generators,

$$V_{\mathcal{S}} = \{\, |\psi\rangle \;:\; S_i\,|\psi\rangle = |\psi\rangle \;\text{ for } i = 1, \ldots, m \,\}\,. \tag{2.10}$$

The condition $-I \notin \mathcal{S}$ guarantees that $V_{\mathcal{S}}$ is non-trivial, and each independent generator halves its dimension, so $V_{\mathcal{S}}$ has dimension $2^{n-m}$ and encodes $k = n - m$ logical qubits.

The generators are exactly the checks that are measured during syndrome measurement. Suppose that an encoded state $|\psi\rangle \in V_{\mathcal{S}}$ suffers a Pauli error $E$. If $E$ commutes with a generator $S_i$, then $S_i E\,|\psi\rangle = E S_i\,|\psi\rangle = E\,|\psi\rangle$, and measuring $S_i$ still returns $+1$. If $E$ anticommutes with $S_i$, then $S_i E\,|\psi\rangle = -E\,|\psi\rangle$, and the outcome flips to $-1$: the error is detected [44, Sec. 10.5.5]. The syndrome thus records which generators anticommute with $E$. Since every code state is a $+1$ eigenstate of every generator, the syndrome carries no information about the encoded state itself.

Errors that commute with all generators go undetected, and they come in two kinds. Elements of $\mathcal{S}$ act trivially on the code space and are harmless. Pauli operators that commute with all of $\mathcal{S}$ but are not themselves in $\mathcal{S}$, up to a phase, map the code space onto itself non-trivially; these are the *logical operators*, and an undetected error of this kind corrupts the encoded information. The minimum weight of a logical operator is the *distance* $d$ of the code, and a code with $n$ physical qubits, $k$ logical qubits and distance $d$ is denoted $[[n, k, d]]$. It detects every error of weight less than $d$ and corrects every error of weight up to $\lfloor (d-1)/2 \rfloor$ [27]. A stabilizer code whose generators each consist either only of $X$ or only of $Z$ operators is a *Calderbank–Shor–Steane (CSS) code* [8, 51], [44, Sec. 10.4.2]. In a CSS code, $X$ errors are detected by the $Z$-type checks and $Z$ errors by the $X$-type checks, independently of each other.

### 2.4.3 The Three-Qubit Repetition Code

The simplest example is the three-qubit repetition code, or bit-flip code [44, Sec. 10.1], which encodes

$$|\overline{0}\rangle = |000\rangle\,, \qquad |\overline{1}\rangle = |111\rangle\,, \tag{2.11}$$

so that $\alpha\,|0\rangle + \beta\,|1\rangle \mapsto \alpha\,|000\rangle + \beta\,|111\rangle$. This is an entangled state, not three copies of the original one, so no-cloning is not violated. The stabilizer group is generated by the two parity checks

$$S_1 = Z_1 Z_2 = ZZI, \qquad S_2 = Z_2 Z_3 = IZZ, \tag{2.12}$$

**Table 2.2:** Syndrome table of the three-qubit repetition code.

| Error | $S_1 = ZZI$ | $S_2 = IZZ$ | Correction |
|---|---|---|---|
| none | $+1$ | $+1$ | none |
| $X_1$ | $-1$ | $+1$ | $X_1$ |
| $X_2$ | $-1$ | $-1$ | $X_2$ |
| $X_3$ | $+1$ | $-1$ | $X_3$ |

so $n = 3$, $m = 2$ and $k = 1$. Each check compares the parity of two neighbouring qubits without revealing their individual values. A single bit flip anticommutes with the checks that contain the flipped qubit, which yields the syndromes in Table 2.2. Each single $X$ error, as well as the absence of an error, has a distinct syndrome, and the correction is to apply the same $X$ again.

Two bit flips, say $X_1X_2$, produce the syndrome of $X_3$; the decoder then applies $X_3$, and the net effect $X_1X_2X_3 = \overline{X}$ is a logical error, so the distance against bit flips is three. Against phase flips, however, the code offers no protection at all: $Z_1$ commutes with both checks and acts as the logical operator $\overline{Z}$, so the distance against $Z$ errors is one. Exchanging the roles of $X$ and $Z$, i.e., encoding into $|{+}{+}{+}\rangle$ and $|{-}{-}{-}\rangle$ with checks $XXI$ and $IXX$, gives the phase-flip code, which corrects $Z$ errors but not $X$ errors. The repetition code thus already shows the asymmetry that codes for biased noise exploit: a code can be strong against one type of error at the expense of another, and under pure dephasing a phase-flip repetition code is all that is needed (Section 2.7.2).

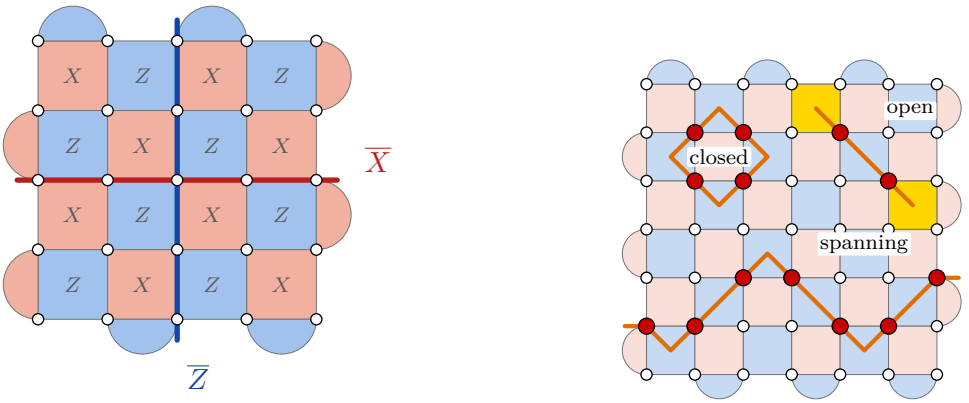
## 2.5 The Surface Code

### 2.5.1 The Rotated Surface Code

The surface code is a stabilizer code whose checks are local on a two-dimensional lattice. It descends from Kitaev's toric code [39] via the planar code with open boundaries [6]. Its *rotated* layout [3, 36], considered in this thesis, is a leading candidate for fault-tolerant quantum computing, because it requires only nearest-neighbour interactions on a square grid and tolerates comparatively high error rates (Section 2.8.3).

The rotated surface code of distance $d$ places $d \times d$ data qubits on the vertices of a square lattice, as shown in Figure 2.1(a). Each face of the lattice, or *plaquette*, carries one check, and the checks alternate between $X$ type and $Z$ type in a checkerboard pattern: an $X$-type plaquette measures $XXXX$ on its four corner qubits, and a $Z$-type plaquette measures $ZZZZ$. The checks thus have weight four. Plaquettes of the same type trivially commute, and neighbouring $X$- and $Z$-type plaquettes share exactly two qubits, so they anticommute on an even number of qubits and commute as well (Section 2.2.1). Since every check is purely $X$ or purely $Z$, the rotated surface code is a CSS code.

The lattice has open boundaries, which are closed off by weight-two checks on every other boundary edge: $Z$-type checks along the top and bottom boundaries and $X$-type checks along the left and right boundaries. Altogether there are $(d-1)^2$ weight-four and $2(d-1)$ weight-two checks, i.e., $d^2 - 1$ independent generators on $d^2$ data qubits, which leaves $k = 1$ logical qubit. Each check is measured with its own ancilla qubit at the centre of its plaquette, so a distance-$d$ patch uses $2d^2 - 1$ physical qubits in total.




(a) (b)

**Figure 2.1:** (a) Distance-5 rotated surface code with data qubits (circles) on the vertices and $X$-type (red) and $Z$-type (blue) checks on the plaquettes and boundaries; ancillas (not shown) sit at the plaquette centres. The logical operators $\overline{X}$ and $\overline{Z}$ cross at a single qubit. (b) Chains of $X$ errors (red) on a distance-7 code, drawn through the $Z$-type checks they flip: a closed chain, an open chain with two detection events (yellow), and a spanning chain, which is a logical error.

A string of $X$ operators along a row of the lattice overlaps every $Z$-type check on either zero or two qubits and therefore commutes with all checks. More generally, a string of $X$ operators commutes with the $Z$-type checks it passes through but anticommutes with those at its endpoints, unless an endpoint lies on the left or right boundary, which carries no $Z$-type checks. Hence the logical operator $\overline{X}$ is a string of $X$ operators connecting the left and right ($X$-type) boundaries, and $\overline{Z}$ is a string of $Z$ operators connecting the top and bottom ($Z$-type) boundaries: a logical operator is a string spanning the lattice between two boundaries of the same type [15, 21]. The two strings run perpendicular to each other and cross an odd number of times—the straight strings in Figure 2.1(a) at exactly one qubit—so $\overline{X}$ and $\overline{Z}$ anticommute, as logical Pauli operators must. The shortest such string has length $d$, so the code distance equals the linear size of the lattice. The rotated surface code is therefore a $[[d^2, 1, d]]$ code, using about half as many data qubits as the unrotated planar code of the same distance.

### 2.5.2 Syndrome Extraction

A check $S = P_1 \otimes P_2 \otimes P_3 \otimes P_4$ with $P_j \in \{X, Y, Z\}$ is measured indirectly with the help of an ancilla, using the circuit in Figure 2.2(a) [44, Sec. 10.5.8]. The ancilla is prepared in the $X$ basis, in $|+\rangle$, and four controlled-Pauli gates, each controlled by the ancilla and targeting one data qubit, together apply a controlled-$S$ operation. If the data qubits are in an eigenstate of $S$ with eigenvalue $s = \pm 1$, the joint state evolves as

$$|+\rangle\,|\psi\rangle \;\mapsto\; \frac{|0\rangle\,|\psi\rangle + |1\rangle\,S\,|\psi\rangle}{\sqrt{2}} = \frac{|0\rangle + s\,|1\rangle}{\sqrt{2}}\;|\psi\rangle\,, \tag{2.13}$$

so the eigenvalue of the generator is written onto the phase of the ancilla, which ends up in $|+\rangle$ for $s = +1$ and in $|-\rangle$ for $s = -1$. Measuring the ancilla in the $X$ basis reads this phase

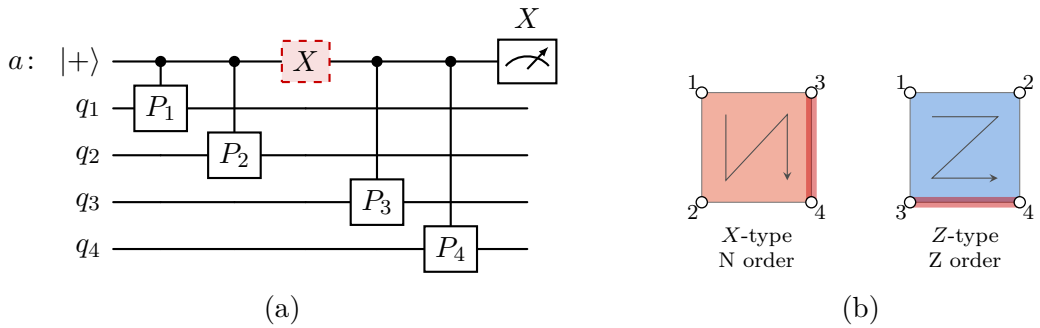



**Figure 2.2:** (a) Measurement of a weight-four check $P_1 \otimes P_2 \otimes P_3 \otimes P_4$ with an ancilla $a$ prepared in $|+\rangle$, four controlled-Pauli gates and an $X$-basis measurement. An $X$ fault on the ancilla at the dashed location propagates through the last two gates into the hook error $P_3 \otimes P_4$ on the data. (b) Gate orders for the two check types. The numbers give the order in which the ancilla interacts with the data qubits, and the shaded edge marks the hook error. With $\overline{X}$ horizontal and $\overline{Z}$ vertical, as in Figure 2.1(a), both hooks are perpendicular to the logical operator of the same type.

back out. For a general state of the data, the measurement projects onto the eigenspace of $S$ with the observed eigenvalue. The outcome is the eigenvalue of the stabilizer generator and nothing more, so the encoded state survives. In the CSS surface code, the controlled gates are CX gates for $X$-type checks and CZ gates for $Z$-type checks; the deformed codes of Section 2.7 also use CY gates.

One round of syndrome extraction measures all $d^2 - 1$ checks in parallel: ancilla preparation, four layers of controlled gates, and ancilla measurement. The order in which each ancilla interacts with its data qubits is a scheduling choice. It is constrained by the rule that a qubit takes part in at most one gate per time step (Section 2.1.2), and by the requirement that, where an $X$-type and a $Z$-type check share qubits, their interleaved gates still measure both checks correctly [21, 54].

Since the measurement circuit is itself noisy, a single round of outcomes cannot be trusted: a faulty ancilla measurement would look like a data error. Syndrome extraction is therefore repeated over many rounds [15], and errors are inferred from *changes* of the outcomes. Formally, a *detector* is a parity of measurement outcomes that is deterministic in the absence of noise, here the product of the outcomes of the same check in two consecutive rounds; in the first and last round, the outcomes are compared with the initial state and with the final data measurement instead (Section 2.8.1). A detector whose value deviates from its noiseless value is a *detection event*. A data error occurring between two rounds causes detection events on the adjacent checks in a single round, whereas a measurement error causes detection events on the same check in two consecutive rounds.

The two-qubit gates also spread errors. An $X$ error on the ancilla is copied by every subsequent controlled-$P_j$ gate onto its data qubit as $P_j$, just as CNOT copies $X_c$ onto the target (Section 2.2.2). A single such fault after the second of the four gates thus leaves the weight-two error $P_3 \otimes P_4$ on the data, a *hook error* [15]. If the schedule aligns the hook errors with the logical operator of the same type, about $d/2$ faults suffice to complete a logical operator, and the circuit-level distance is roughly halved. Ordering the gates

such that the hooks of $X$-type checks run perpendicular to $\overline{X}$ and those of $Z$-type checks perpendicular to $\overline{Z}$, as in the N- and Z-shaped orders of Figure 2.2(b), preserves the full distance [54].

## 2.6 Decoding

### 2.6.1 The Decoding Problem

The *decoder* is the classical algorithm that turns measurement data into a correction. Its input is the set of detection events of one run of the experiment, and its output is a guess at the logical class of the error—in a memory experiment, simply whether the logical observable has been flipped [53]. A guess at the exact error is unnecessary. Many distinct errors produce the same detection events, and two errors that differ by a stabilizer, $E' = ES$ with $S \in \mathcal{S}$, act identically on the code space. Errors therefore fall into classes whose members differ by stabilizers, and only the class matters: the optimal, *maximum-likelihood* decoder chooses the most likely class, summing the probabilities of all errors in it, rather than the most likely individual error [15, 5]. This *degeneracy* has no counterpart in classical coding.

With repeated noisy measurements, decoding becomes a problem in space-time [15, 21]. It is represented by a *decoding graph* whose nodes are the detectors (Section 2.5.2) and whose edges are the error mechanisms that flip a pair of detectors. A data error between two rounds flips neighbouring checks in the same round and becomes an edge in the space direction; a measurement error flips the same check in two consecutive rounds and becomes an edge in the time direction. Mechanisms that flip a single detector, such as errors on certain qubits at the boundary, are connected to an additional boundary node. Each edge $e$ carries the probability $p_e$ of its mechanism and the weight $w_e = \log((1-p_e)/p_e)$, so that likely errors are cheap and the most likely error consistent with the detection events is the one of minimum total weight. Circuit-level noise adds further edges, for example diagonal ones caused by hook errors. In practice, the decoding graph need not be constructed by hand: Stim derives it automatically from the noisy circuit as a *detector error model* [22].

### 2.6.2 Error Chains

In the decoding graph, an error is a set of edges, an *error chain*, and its detection events are the endpoints of the chain, i.e., the nodes at which an odd number of its edges meet [15]. Three kinds of chains can be distinguished, illustrated in Figure 2.1(b) for $X$ errors within a single round:

- A *closed chain* forms a loop without endpoints and produces no detection events. It is a product of stabilizer generators—the smallest example is a single check, whose four qubits encircle a plaquette of the other type—and therefore acts trivially on the code space: it cannot be detected, but it is harmless.
- An *open chain* has two endpoints, which appear as detection events. It is a detectable error, and the decoder's task is to pair up the endpoints with a correction chain.
- A *spanning chain* stretches from one boundary of the lattice to the opposite boundary of the same type. Its endpoints lie on the boundaries, so it produces no detection events, yet it is a logical operator: it cannot be detected, and it is harmful—a logical error.

After correction, the error chain and the correction chain together form closed or spanning chains, and decoding fails exactly when their combination contains a spanning chain. This happens, for example, when the decoder pairs the endpoints of a long open chain the "wrong way", with the boundaries rather than with each other. Since a spanning chain consists of at least $d$ edges, while the decoder chooses the shorter of the two completions, an error chain needs at least about $d/2$ edges to cause a logical failure.

### 2.6.3 Decoders

Decoders differ in accuracy and speed, and the choice affects both the observed logical error rates and the feasibility of decoding in real time [14]. Three widely used decoders illustrate this trade-off.

The *minimum-weight perfect matching* (MWPM) decoder pairs up the detection events, with each other or with the boundary, such that the total weight of the connecting paths is minimal. This is a minimum-weight perfect matching problem on the decoding graph, which Edmonds' blossom algorithm solves exactly in polynomial time [17, 18]. Proposed for the surface code by Dennis et al. [15], MWPM is the standard baseline for the surface code, not least because of fast implementations such as PyMatching [33] and its sparse blossom algorithm [34]. It finds the most likely error rather than the most likely class. Moreover, since every edge connects only two nodes, it cannot represent errors that flip more than two detectors, such as $Y$ errors in the CSS code; these are decomposed into independent edges, and their correlations are lost.

The *union-find* decoder grows clusters around the detection events until every cluster contains an even number of events or touches the boundary, and then finds a correction within each cluster [13]. Its running time is almost linear in the size of the decoding graph, at the cost of a somewhat lower accuracy than MWPM.

The *belief-matching* decoder first runs belief propagation on the full error model, including the hyperedges that a matching graph cannot represent, and uses the resulting marginal error probabilities to reweight the edges of the decoding graph before running MWPM [11, 35]. It is more accurate than MWPM, in particular for correlated and biased noise, but considerably slower.

## 2.7 Clifford Deformed Surface Codes

### 2.7.1 Definition

A Clifford-deformed surface code is obtained by applying a single-qubit Clifford operation $C_j$ to a subset of the data qubits $j$ of the surface code [16, 4]. Under the deformation $U = \bigotimes_j C_j$, the stabilizer group $\mathcal{S}$ becomes $U\mathcal{S}U^\dagger$ and the code space becomes $UV_\mathcal{S}$. Every check is conjugated qubit by qubit, and by Table 2.1, $C_j$ simply *relabels* the Pauli operator acting on qubit $j$; the Hadamard gate, for instance, exchanges $X$ and $Z$, so that a check $XXXX$ may become $XZZX$. Signs created by the conjugation can be absorbed into the definition of the generators, so a deformation is fully specified by one of the six relabelings for each data qubit. Since $U$ is a product of single-qubit unitaries, it preserves the weight of every Pauli operator, and the deformed code is again a $[[d^2, 1, d]]$ code. What changes is *which* physical errors are detected by which checks, and hence which errors are likely to combine into logical errors.

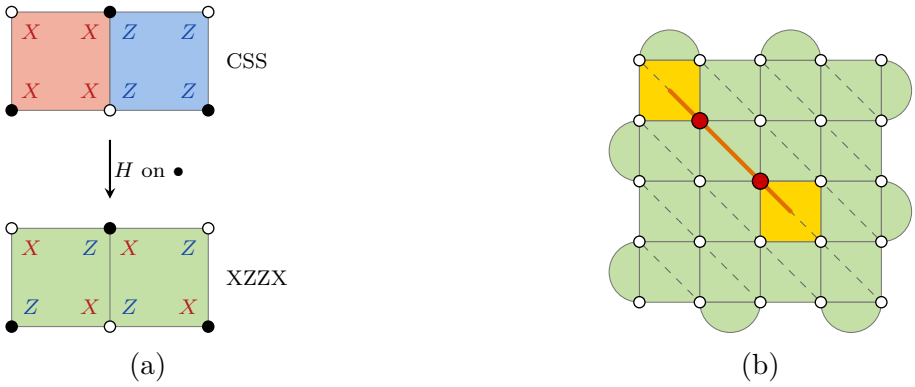



**Figure 2.3:** (a) Applying $H$ to the data qubits of one checkerboard sublattice (black) maps both the $X$-type check $XXXX$ and the $Z$-type check $ZZZZ$ of the CSS code onto the form $XZZX$, with $X$ on one diagonal of each plaquette and $Z$ on the other. (b) In the distance-5 XZZX code, a $Z$ error flips only the checks that carry $X$ on its qubit, which lie on a common diagonal; the chain of two $Z$ errors (red) produces detection events (yellow) only at its ends. Under pure dephasing, every diagonal (dashed) is an independent repetition code.

The experiment transforms along with the code. Each data qubit is prepared and finally measured in the eigenbasis of its relabeled Pauli—for example, in the $X$ basis instead of the $Z$ basis on a qubit to which $H$ is applied. With this, the time-boundary detectors change accordingly: they are formed by the checks whose outcomes are deterministic in the first round and recoverable from the final data measurement. In the syndrome-extraction circuit, the controlled gate on each leg follows the new Pauli: CX, CY or CZ for $X$, $Y$ or $Z$, respectively [4, 12]. The ancillas, their preparation in $|+\rangle$ and their $X$-basis measurement are unchanged. The lattice, the qubit count and the gate schedule are untouched as well, so a deformation changes only which Pauli each check measures on each qubit and requires neither additional qubits nor additional time steps.

### 2.7.2 Biased Noise and Necessity of Clifford Deformation

The CSS surface code treats $X$ and $Z$ errors symmetrically: half of its checks detect $X$ errors, the other half detect $Z$ errors, and its distance against both is $d$. Under biased noise, this symmetry is wasteful. The dominant $Z$ errors are detected only by the $X$-type checks, while the $Z$-type checks guard against errors that rarely occur. Under pure dephasing, the CSS code reduces to a code against $Z$ errors alone and, for code-capacity noise, has the same threshold of about 10% as under bit-flip noise, far below the theoretical limit of 50% for pure dephasing [55, 4]. Clifford deformations offer a way to tailor the code to the noise without changing the hardware layout [56, 16]. Two prominent examples are the XZZX code and the XY code.

**XZZX code.** Applying $H$ to the data qubits of one checkerboard sublattice turns every check, of either type, into the form $XZZX$, as shown in Figure 2.3(a) [4]. In every plaquette, the two qubits on one diagonal carry $X$ and the two on the other diagonal carry $Z$. A $Z$ error anticommutes only with the checks that carry $X$ on its qubit, and

these lie along one diagonal direction of the lattice; an $X$ error flips checks along the other diagonal. Under pure dephasing, detection events therefore move only along diagonals, and the code decouples into independent repetition codes, one per diagonal (Figure 2.3(b)). As the bias increases, the threshold of the XZZX code grows, approaching the 50% limit for pure dephasing with code-capacity noise, while under depolarizing noise it performs as well as the CSS code; the advantage under biased noise persists at circuit level [4, 12].

**XY code.** Applying $\sqrt{X}$ to every data qubit leaves $X$ unchanged but turns $Z$ into $Y$ (Table 2.1), so the $Z$-type checks become $Y$-type checks $YYYY$ [55, 56]. A $Z$ error now anticommutes with both the $X$- and the $Y$-type checks, so all checks contribute to detecting the dominant error. Equivalently, since the deformation can be moved from the code onto the noise, the XY code under $Z$-biased noise is the CSS code under $Y$-biased noise, the setting in which surface codes were first shown to reach ultrahigh thresholds under strong bias [55, 57].

**The family of deformations.** Choosing one of the six single-qubit Cliffords of Table 2.1 for each of the $n$ data qubits yields a family of $6^n$ codes, which contains the CSS, XZZX and XY codes as special cases. Many members of this family are equivalent because of symmetries of the noise [16, Sec. II]. Under $Z$-biased noise with $p_X = p_Y$, exchanging $X$ and $Y$ on a qubit leaves the noise invariant, so it only matters which Pauli of the original checks is relabeled into $Z$ on each qubit; three deformations per qubit suffice, and $3^n$ inequivalent classes remain. Under depolarizing noise, every relabeling is a symmetry, and all $6^n$ codes form a single class. These equivalences hold exactly for code-capacity noise; at circuit level, a deformation also changes the two-qubit gates and the noise they produce, which can break them. Which deformation performs best can depend on the bias, but also on the figure of merit, such as the threshold or the logical error rate below threshold, and on the decoder; a deformed code may, for instance, have boundaries that are fragile under a matching decoder [35]. Comparing such deformations systematically is the purpose of the framework developed in this thesis.

## 2.8 Memory Experiment and Quantifying Results

### 2.8.1 Memory Experiment

The *memory experiment* measures how well a code stores a logical qubit, and it is the standard benchmark of quantum error correction in both simulations and hardware [15, 25]. A single run, or *shot*, consists of four steps:

1. *Preparation:* the data qubits are prepared in a product state that is an eigenstate of one logical operator, for example $|0\rangle^{\otimes d^2}$, which is a $+1$ eigenstate of $\overline{Z}$ and of every $Z$-type check.
2. *Syndrome extraction:* $R$ rounds of the circuit of Section 2.5.2 are executed.
3. *Readout:* all data qubits are measured in the preparation basis. The final values of the checks of the corresponding type, as well as the value of the logical operator, are computed from the outcomes.

4. *Decoding:* the decoder predicts from the detection events whether the logical observable has been flipped. The shot succeeds if the prediction is correct and fails otherwise.

In the first round, only the checks of which the initial state is an eigenstate have deterministic outcomes, and the final data measurement determines the values of the same checks once more; these provide the *time-boundary detectors*.

In a $Z$ *memory*, the logical state $|\overline{0}\rangle$ is prepared and $\overline{Z}$ is read out, so the experiment tests the protection against the $X$ errors that flip $\overline{Z}$. An $X$ *memory* prepares $|\overline{+}\rangle$, reads out $\overline{X}$ and tests the protection against $Z$ errors [54, 25]. For the CSS code under depolarizing noise, the two behave almost identically by symmetry, but under biased noise, and for deformed codes, the $X$ and $Z$ memories of the same code can behave very differently, so both have to be examined. The number of rounds is conventionally chosen proportional to the distance, typically $R = d$. The decoding graph then extends as far in time as in space, which balances the protection against measurement errors with that against data errors [15].

### 2.8.2 Logical Error Rate

Each shot of a memory experiment is a Bernoulli trial that fails with probability $p_L$, the *logical error rate*. With $k$ failures in $N$ shots, $p_L$ is estimated as $\hat{p}_L = k/N$. Its statistical uncertainty is reported as a confidence interval. The textbook Wald interval $\hat{p}_L \pm z\sqrt{\hat{p}_L(1-\hat{p}_L)/N}$ behaves poorly for small $\hat{p}_L$, which is exactly the regime of interest for good codes: for $k = 0$, it even collapses to a single point. The *Wilson score interval* [61]

$$p_\pm = \frac{1}{1+z^2/N}\left(\hat{p}_L + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{p}_L(1-\hat{p}_L)}{N} + \frac{z^2}{4N^2}}\right), \tag{2.14}$$

where $z$ is the standard normal quantile ($z \approx 1.96$ for 95% confidence), remains well behaved near zero and is the recommended alternative [7].

The per-shot rate $p_L$ depends on the number of rounds: a longer experiment has more opportunities to fail. To compare runs with different $R$—and, since $R = d$, runs of different distances—$p_L$ is converted into a logical error rate *per round*, $\epsilon$. If each of the $R$ rounds independently flips the logical observable with probability $\epsilon$, the observable is flipped at the end with probability $p_L = \left(1 - (1-2\epsilon)^R\right)/2$, which inverts to [25]

$$\epsilon = \frac{1}{2}\left(1 - (1-2p_L)^{1/R}\right). \tag{2.15}$$

For small error rates, $\epsilon \approx p_L/R$.

### 2.8.3 Threshold

The *threshold* $p_{\mathrm{th}}$ is the physical error rate below which increasing the code distance decreases the logical error rate [15, 29]. Below $p_{\mathrm{th}}$, adding qubits helps, and the logical error rate can be made arbitrarily small by increasing $d$; above $p_{\mathrm{th}}$, larger codes perform worse, because they offer more opportunities for errors than they can correct. When $p_L$ is plotted against $p$ for several distances, the curves therefore cross at $p_{\mathrm{th}}$.

The threshold can be mapped to a phase transition in statistical mechanics. For code-capacity noise, optimal decoding of the surface code corresponds to the random-bond

Ising model, and with noisy measurements to a three-dimensional random-plaquette gauge model; the threshold corresponds to the transition between their ordered and disordered phases [15, 59]. The mapping explains the sharp crossing and motivates a finite-size scaling analysis borrowed from statistical physics. Near the threshold, $p_L$ is assumed to depend on $p$ and $d$ only through the rescaled variable

$$x = (p - p_{\mathrm{th}})\, d^{1/\nu}, \tag{2.16}$$

with a critical exponent $\nu$, and is fitted by a low-order polynomial, $p_L \approx A + Bx + Cx^2$ [59, 55]. Fitting this ansatz to data from several distances yields $p_{\mathrm{th}}$ together with an uncertainty estimate.

A threshold is not a property of a code alone: it belongs jointly to the code, the circuit, the noise model with its parameters, such as the bias $\eta$, and the decoder [52]. For the surface code, the threshold is about 10% for code-capacity bit-flip noise [15, 59] but only of the order of 1% for circuit-level depolarizing noise [60, 21]. Thresholds of different codes are therefore comparable only when all other components are fixed.

### 2.8.4 Suppression and Teraquop Footprint

Below threshold, the logical error rate per round decreases exponentially with the distance,

$$\epsilon_d \propto \Lambda^{-(d+1)/2}, \tag{2.17}$$

because a logical error requires at least $(d+1)/2$ faults [21, 26]. The *error suppression factor* $\Lambda = \epsilon_d/\epsilon_{d+2}$ is the factor by which the logical error rate drops when the distance grows by two: it is the return on adding qubits. $\Lambda > 1$ indicates operation below threshold, and $\Lambda$ grows as the physical error rate decreases, roughly as $\Lambda \approx p_{\mathrm{th}}/p$ [21]. For comparison, a recent superconducting-qubit experiment demonstrated $\Lambda \approx 2.14$ [26].

Taking the logarithm of (2.17) gives $\log \epsilon_d = c - \frac{d+1}{2} \log \Lambda$, so $\Lambda$ is read off from the slope of a linear fit of $\log \epsilon_d$ against $(d+1)/2$ over several distances [43, Sec. 3.4]. Since $(d+1)/2$ is an affine function of $d$, a fit against odd $d$ gives the same line, with slope $-\frac{1}{2} \log \Lambda$.

The fit also allows an extrapolation to the distances needed in practice. The *teraquop footprint* is the number of physical qubits needed for a logical qubit that survives a trillion logical operations, operationalized as a $d$-round memory experiment with a logical error rate of $10^{-12}$ [24], [43, Sec. 3.4]. It is obtained by extrapolating the fit to the distance at which the $d$-round logical error rate, approximately $d\,\epsilon_d$, reaches $10^{-12}$, rounding up to the next odd $d$, and counting the qubits of that patch, $2d^2 - 1$ for the rotated surface code. The teraquop footprint condenses the performance of a code below threshold into a single, practically meaningful number.

# Chapter 3

# Design and Implementation

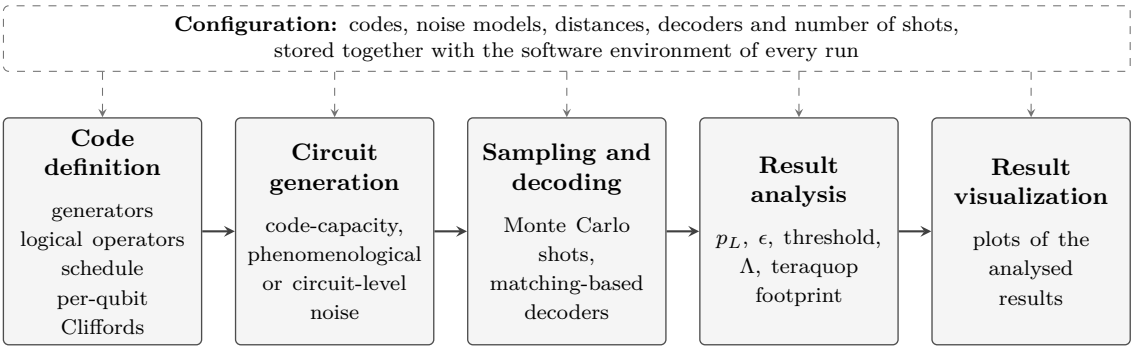



**Figure 3.1:** Pipeline of the framework. A single configuration describes a study. From the code definition, the framework generates the memory-experiment circuits for the chosen noise level, samples and decodes them, analyses the results and visualizes them as plots.

# Chapter 4

# Evaluation

## 4.1 Depolarizing Code-Capacity Threshold

Reproduce the results of [11, 10].

## 4.2 Phenomenological Threshold against Stim

Reproduce the results of [22, 41].

## 4.3 Phenomenological Threshold under Biased Noise

Reproduce the results of [4, 10].

## 4.4 Depolarizing Circuit-Level Threshold

Reproduce the results of [52, 45, 21, 37, 35, 24, 32, 19].

## 4.5 Below-Threshold Suppression and Teraquop Footprint

Reproduce the results of [45].

## 4.6 Decoder Comparison at Code Capacity

Reproduce the results of [11, 41, 13, 14, 10].

# Chapter 5

# Discussion

# Chapter 6

# Conclusion and Future Work

# Acknowledgements

# Bibliography


[1] S. Aaronson and D. Gottesman. Improved simulation of stabilizer circuits. *Phys. Rev. A*, 70:052328, 2004. DOI: 10.1103/PhysRevA.70.052328. arXiv:quant-ph/0406196.

[2] P. Aliferis and J. Preskill. Fault-tolerant quantum computation against biased noise. *Phys. Rev. A*, 78:052331, 2008. DOI: 10.1103/PhysRevA.78.052331. arXiv:0710.1301.

[3] H. Bombin and M. A. Martin-Delgado. Optimal resources for topological two-dimensional stabilizer codes: Comparative study. *Phys. Rev. A*, 76:012305, 2007. DOI: 10.1103/PhysRevA.76.012305. arXiv:quant-ph/0703272.

[4] J. P. Bonilla Ataides, D. K. Tuckett, S. D. Bartlett, S. T. Flammia, and B. J. Brown. The XZZX surface code. *Nat. Commun.*, 12:2172, 2021. DOI: 10.1038/s41467-021-22274-1. arXiv:2009.07851.

[5] S. Bravyi, M. Suchara, and A. Vargo. Efficient algorithms for maximum likelihood decoding in the surface code. *Phys. Rev. A*, 90:032326, 2014. DOI: 10.1103/PhysRevA.90.032326. arXiv:1405.4883.

[6] S. B. Bravyi and A. Yu. Kitaev. Quantum codes on a lattice with boundary. arXiv:quant-ph/9811052, 1998.

[7] L. D. Brown, T. T. Cai, and A. DasGupta. Interval estimation for a binomial proportion. *Statist. Sci.*, 16:101, 2001. DOI: 10.1214/ss/1009213286.

[8] A. R. Calderbank and P. W. Shor. Good quantum error-correcting codes exist. *Phys. Rev. A*, 54:1098, 1996. DOI: 10.1103/PhysRevA.54.1098. arXiv:quant-ph/9512032.

[9] A. R. Calderbank, E. M. Rains, P. W. Shor, and N. J. A. Sloane. Quantum error correction via codes over GF(4). *IEEE Trans. Inf. Theory*, 44:1369, 1998. DOI: 10.1109/18.681315. arXiv:quant-ph/9608006.

[10] J. A. Campos and K. R. Brown. Clifford-deformed compass codes. *Quantum*, 10: 2073, 2026. DOI: 10.22331/q-2026-04-16-2073. arXiv:2412.03808.

[11] B. Criger and I. Ashraf. Multi-path summation for decoding 2D topological codes. *Quantum*, 2:102, 2018. DOI: 10.22331/q-2018-10-19-102. arXiv:1709.02154.

[12] A. S. Darmawan, B. J. Brown, A. L. Grimsmo, D. K. Tuckett, and S. Puri. Practical quantum error correction with the XZZX code and Kerr-cat qubits. *PRX Quantum*, 2:030345, 2021. DOI: 10.1103/PRXQuantum.2.030345. arXiv:2104.09539.

[13] N. Delfosse and N. H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021. DOI: 10.22331/q-2021-12-02-595. arXiv:1709.06218.

[14] A. deMarti iOlius, P. Fuentes, R. Orús, P. M. Crespo, and J. Etxezarreta Martinez. Decoding algorithms for surface codes. *Quantum*, 8:1498, 2024. DOI: 10.22331/q-2024-10-10-1498. arXiv:2307.14989.

[15] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill. Topological quantum memory. *J. Math. Phys.*, 43:4452, 2002. DOI: 10.1063/1.1499754. arXiv:quant-ph/0110143.

[16] A. Dua, A. Kubica, L. Jiang, S. T. Flammia, and M. J. Gullans. Clifford-deformed surface codes. *PRX Quantum*, 5:010347, 2024. DOI: 10.1103/PRXQuantum.5.010347. arXiv:2201.07802.

[17] J. Edmonds. Paths, trees, and flowers. *Can. J. Math.*, 17:449, 1965. DOI: 10.4153/CJM-1965-045-4.

[18] J. Edmonds. Maximum matching and a polyhedron with 0,1-vertices. *J. Res. Natl. Bur. Stand. B*, 69B:125, 1965. DOI: 10.6028/jres.069B.013.

[19] J. Etxezarreta Martinez, P. Schnabl, J. Oliva del Moral, R. Dastbasteh, P. M. Crespo, and R. M. Otxoa. Leveraging biased noise for more efficient quantum error correction at the circuit level with two-level qubits. *Phys. Rev. Appl.*, 25:014021, 2026. DOI: 10.1103/q7w6-nljp. arXiv:2505.17718.

[20] R. P. Feynman. Simulating physics with computers. *Int. J. Theor. Phys.*, 21:467, 1982. DOI: 10.1007/BF02650179.

[21] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86:032324, 2012. DOI: 10.1103/PhysRevA.86.032324. arXiv:1208.0928.

[22] C. Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021. DOI: 10.22331/q-2021-07-06-497. arXiv:2103.02202.

[23] C. Gidney and M. Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. DOI: 10.22331/q-2021-04-15-433. arXiv:1905.09749.

[24] C. Gidney, M. Newman, A. Fowler, and M. Broughton. A fault-tolerant honeycomb memory. *Quantum*, 5:605, 2021. DOI: 10.22331/q-2021-12-20-605. arXiv:2108.10457.

[25] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614:676, 2023. DOI: 10.1038/s41586-022-05434-1. arXiv:2207.06431.

[26] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638:920, 2025. DOI: 10.1038/s41586-024-08449-y. arXiv:2408.13687.

[27] D. Gottesman. *Stabilizer Codes and Quantum Error Correction.* PhD thesis, California Institute of Technology, 1997. DOI: 10.7907/rzr7-dt72. arXiv:quant-ph/9705052.

[28] D. Gottesman. The Heisenberg representation of quantum computers. In *Group22: Proceedings of the XXII International Colloquium on Group Theoretical Methods in Physics*, pages 32–43, Cambridge, MA, 1999. International Press. arXiv:quant-ph/9807006.

[29] D. Gottesman. An introduction to quantum error correction and fault-tolerant quantum computation. *Proc. Symp. Appl. Math.*, 68:13, 2010. DOI: 10.1090/psapm/068/2762145. arXiv:0904.2557.

[30] L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996. DOI: 10.1145/237814.237866. arXiv:quant-ph/9605043.

[31] M. B. Hastings and J. Haah. Dynamically generated logical qubits. *Quantum*, 5:564, 2021. DOI: 10.22331/q-2021-10-19-564. arXiv:2107.02194.

[32] B. Hetényi and J. R. Wootton. Tailoring quantum error correction to spin qubits. *Phys. Rev. A*, 109:032433, 2024. DOI: 10.1103/PhysRevA.109.032433. arXiv:2306.17786.

[33] O. Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Trans. Quantum Comput.*, 3:16, 2022. DOI: 10.1145/3505637. arXiv:2105.13082.

[34] O. Higgott and C. Gidney. Sparse Blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025. DOI: 10.22331/q-2025-01-20-1600. arXiv:2303.15933.

[35] O. Higgott, T. C. Bohdanowicz, A. Kubica, S. T. Flammia, and E. T. Campbell. Improved decoding of circuit noise and fragile boundaries of tailored surface codes. *Phys. Rev. X*, 13:031007, 2023. DOI: 10.1103/PhysRevX.13.031007. arXiv:2203.04948.

[36] C. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter. Surface code quantum computing by lattice surgery. *New J. Phys.*, 14:123011, 2012. DOI: 10.1088/1367-2630/14/12/123011. arXiv:1111.4022.

[37] S. Huang, M. Newman, and K. R. Brown. Fault-tolerant weighted union-find decoding on the toric code. *Phys. Rev. A*, 102:012419, 2020. DOI: 10.1103/PhysRevA.102.012419. arXiv:2004.04693.

[38] P. Iyer and D. Poulin. Hardness of decoding quantum stabilizer codes. *IEEE Trans. Inf. Theory*, 61:5209, 2015. DOI: 10.1109/TIT.2015.2422294. arXiv:1310.3235.

[39] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Ann. Phys.*, 303:2, 2003. DOI: 10.1016/S0003-4916(02)00018-0. arXiv:quant-ph/9707021.

[40] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver. A quantum engineer's guide to superconducting qubits. *Appl. Phys. Rev.*, 6:021318, 2019. DOI: 10.1063/1.5089550. arXiv:1904.06560.

[41] T.-H. Lin and C.-Y. Lai. Union-intersection union-find for decoding depolarizing errors in topological codes. *IEEE J. Sel. Areas Inf. Theory*, 6:163, 2025. DOI: 10.1109/JSAIT.2025.3581810. arXiv:2506.14745.

[42] S. Lloyd. Universal quantum simulators. *Science*, 273:1073, 1996. DOI: 10.1126/science.273.5278.1073.

[43] M. McEwen, D. Bacon, and C. Gidney. Relaxing hardware requirements for surface code circuits using time-dynamics. *Quantum*, 7:1172, 2023. DOI: 10.22331/q-2023-11-07-1172. arXiv:2302.02192.

[44] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010. DOI: 10.1017/CBO9780511976667.

[45] A. R. O'Rourke and S. Devitt. Compare the pair: Rotated versus unrotated surface codes at equal logical error rates. *Phys. Rev. Res.*, 7:033074, 2025. DOI: 10.1103/PhysRevResearch.7.033074. arXiv:2409.14765.

[46] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. DOI: 10.22331/q-2018-08-06-79. arXiv:1801.00862.

[47] S. Puri et al. Bias-preserving gates with stabilized cat qubits. *Sci. Adv.*, 6:eaay5901, 2020. DOI: 10.1126/sciadv.aay5901. arXiv:1905.00450.

[48] P. W. Shor. Scheme for reducing decoherence in quantum computer memory. *Phys. Rev. A*, 52:R2493, 1995. DOI: 10.1103/PhysRevA.52.R2493.

[49] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26:1484, 1997. DOI: 10.1137/S0097539795293172. arXiv:quant-ph/9508027.

[50] A. M. Steane. Error correcting codes in quantum theory. *Phys. Rev. Lett.*, 77:793, 1996. DOI: 10.1103/PhysRevLett.77.793.

[51] A. M. Steane. Multiple-particle interference and quantum error correction. *Proc. R. Soc. Lond. A*, 452:2551, 1996. DOI: 10.1098/rspa.1996.0136. arXiv:quant-ph/9601029.

[52] A. M. Stephens. Fault-tolerant thresholds for quantum error correction with the surface code. *Phys. Rev. A*, 89:022321, 2014. DOI: 10.1103/PhysRevA.89.022321. arXiv:1311.5003.

[53] B. M. Terhal. Quantum error correction for quantum memories. *Rev. Mod. Phys.*, 87:307, 2015. DOI: 10.1103/RevModPhys.87.307. arXiv:1302.3428.

[54] Y. Tomita and K. M. Svore. Low-distance surface codes under realistic quantum noise. *Phys. Rev. A*, 90:062320, 2014. DOI: 10.1103/PhysRevA.90.062320. arXiv:1404.3747.

[55] D. K. Tuckett, S. D. Bartlett, and S. T. Flammia. Ultrahigh error threshold for surface codes with biased noise. *Phys. Rev. Lett.*, 120:050505, 2018. DOI: 10.1103/PhysRevLett.120.050505. arXiv:1708.08474.

[56] D. K. Tuckett, A. S. Darmawan, C. T. Chubb, S. Bravyi, S. D. Bartlett, and S. T. Flammia. Tailoring surface codes for highly biased noise. *Phys. Rev. X*, 9:041031, 2019. DOI: 10.1103/PhysRevX.9.041031. arXiv:1812.08186.

[57] D. K. Tuckett, S. D. Bartlett, S. T. Flammia, and B. J. Brown. Fault-tolerant thresholds for the surface code in excess of 5% under biased noise. *Phys. Rev. Lett.*, 124:130501, 2020. DOI: 10.1103/PhysRevLett.124.130501. arXiv:1907.02554.

[58] J. J. Wallman and J. Emerson. Noise tailoring for scalable quantum computation via randomized compiling. *Phys. Rev. A*, 94:052325, 2016. DOI: 10.1103/PhysRevA.94.052325. arXiv:1512.01098.

[59] C. Wang, J. Harrington, and J. Preskill. Confinement-Higgs transition in a disordered gauge theory and the accuracy threshold for quantum memory. *Ann. Phys.*, 303:31, 2003. DOI: 10.1016/S0003-4916(02)00019-2. arXiv:quant-ph/0207088.

[60] D. S. Wang, A. G. Fowler, and L. C. L. Hollenberg. Surface code quantum computing with error rates over 1%. *Phys. Rev. A*, 83:020302(R), 2011. DOI: 10.1103/PhysRevA.83.020302. arXiv:1009.3686.

[61] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.*, 22:209, 1927. DOI: 10.1080/01621459.1927.10502953.

[62] W. K. Wootters and W. H. Zurek. A single quantum cannot be cloned. *Nature*, 299: 802, 1982. DOI: 10.1038/299802a0.